

Will it Snow on Thanksgiving?

Project 1

Alec Krsek
aleckrsek@oregonstate.edu

October 4th, 2026

Introduction

In this project, we use the Monte-Carlo simulation and a Box-Muller transform to determine the probability that it will snow on Thanksgiving in Bend, Oregon. To make this program even faster, we utilize OpenMP to parallelize our Monte Carlo simulation. Our simulation uses the first-snow day in Bend, being November 9th, and we apply a standard deviation and box-muller transform to get the simulated day. Any dates outside of Bends historical snow-fall range (September 24 to May 23) will be discarded. The trial loop in the code is parallelized, while the initial random number generation is serial.

Workstation Specifications

Operating System: Fedora Linux (Kernel 7.2.9) on baremetal

CPU Name: Ryzen 7 9700X

Cores: 8

Threads per core: 2

RAM: 64GB DDR5 (6000 MHz)

Max Clock Speed: 5.6 GHz (Max I can hit before thermal throttling)

Cache:

L1d: 384 KiB

L1i: 256 KiB

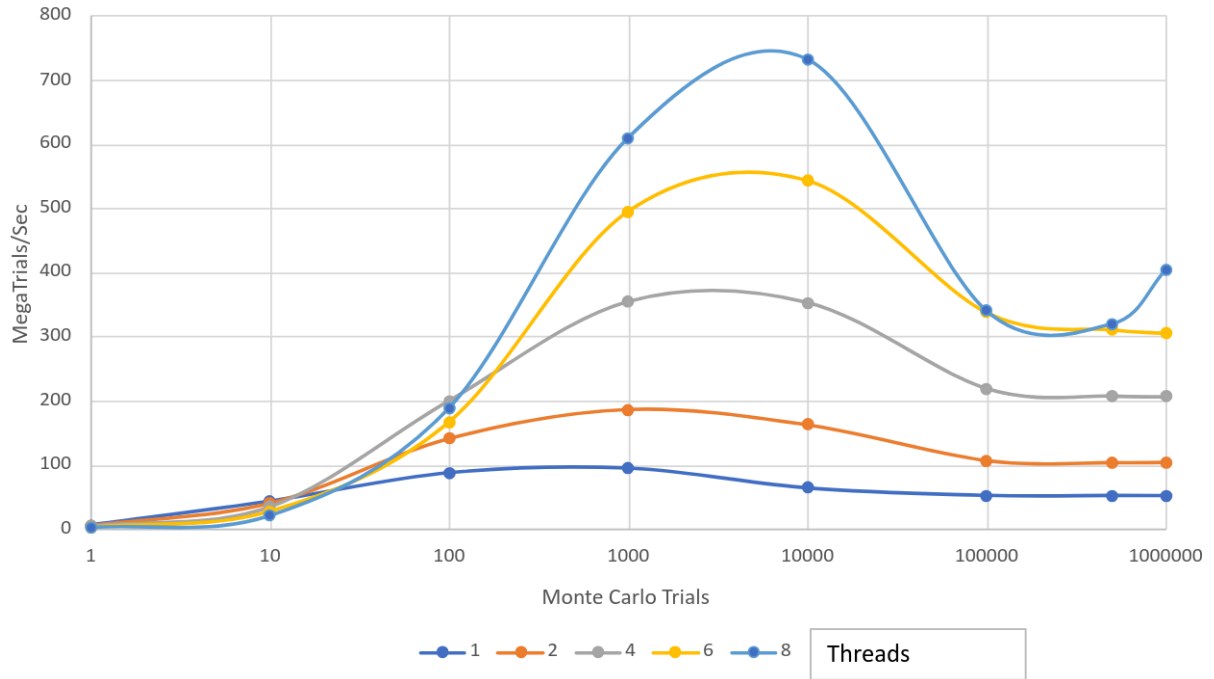
L2: 8 MiB

L3: 32 MiB

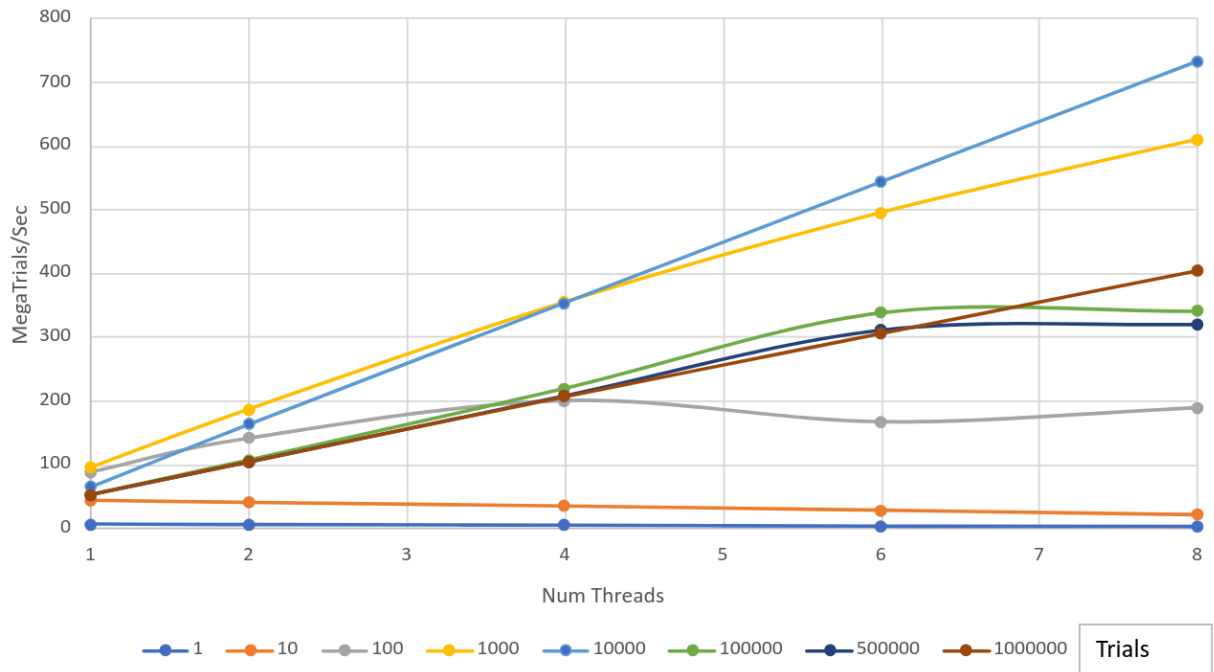
Results

Sum of MegaTrials/Sec	Num Trials								
Threads		1	10	100	1000	10000	100000	500000	1000000
1		6.25	43.48	87.72	94.88	63.96	52.4	52.22	52.08
2		5.26	40	140.85	186.22	162.26	106.58	103.35	103.67
4		4.35	34.48	200	354.61	353.11	218.84	207.09	205.8
6		2.86	27.78	166.67	495.05	542.89	337.43	310.12	304.89
8		2.38	20.83	188.68	609.76	732.06	339.88	319.46	403.41

Performance across Threads



Performance across Threads



In our first graph, we can see the same trend as we have in in-class activities and in our first project. In the shorter number of trials, such as 1 and 10, we see that using one thread is faster than utilizing parallelization. But after we have more trials, we start to see our performance increase. This is because there is overhead when creating new threads. Just creating the threads and utilizing them takes more time than by simply running a single trial with one thread.

We also see that our performance peaks between 1000 and 10,000 trials, goes down and plateaus off. Based on some research, this trend is because of our CPU cache. At the 10,000 trial mark, the amount of data is small enough to fit in our L1 cache, which is the closest memory location for our CPU. This uses around 80 KB of data, so when we increase the amount of trials to around 40,000 trials, our L1 cache will become full. The rest of the data will be stored in L2 and L3 caches, which are physically farther away from the cores on the chip which gives a slower speed.

The first graph also has a weird section between 100,000 and 500,000 threads. We see that our 8 thread performance is essentially the same as 6 threads until the 1,000,000 trial mark. Since my CPU has 8 cores, other cores still had other things to process such as the operating system and any background tasks.

The second graph shows us a lot of the same things that we interpreted from the first graph. If we use 4 or more threads and use 10,000 trials, this configuration will perform the best if performance is all we are after. Just like what we expected due to the L1 cache size.

Probability and Parallel Fraction

Our probability that it will snow in Bend for Thanksgiving this year is ~1.25%

Our parallel fraction can be calculated as such:

$$F_{\text{parallel}} = \frac{n}{n-1} \times \frac{\text{Speedup} - 1}{\text{Speedup}}$$

$$\text{Speedup}_8 = \frac{403.41}{52.08} = 7.74$$

$$\frac{8}{8-1} \times \frac{(7.74 - 1)}{7.74} = 0.995$$

Thus our parallel fraction is 0.995, not bad!
The serial fraction would be 0.005.

What would the speedup be if we had 100 cores?

$$S_{100} = \frac{1}{(1 - F_p) + \frac{F_p}{100}}$$

$$\frac{F_p}{100} = \frac{0.995}{100} = 0.00995$$

$$S_{100} = \frac{1}{0.005 +}$$

$$S = \frac{1}{0.01495} \approx 66.9$$

We would have a speedup of just under 70, given 100 cores. Definitely not as good as our speedup from 1 to 8 cores.

Conclusion

With our Monte Carlo simulation and Box-Muller transform, we calculated that there is a 1.25% chance of snowfall on Thanksgiving this year in Bend. Even though filling in the two arrays with random numbers is a serial process, the Monte Carlo trials were great for parallelization. Getting a parallel fraction of 99.5% shows that we can parallelize this algorithm and gain more performance. Throughout the graphs, it shows that parallelism does work, given you actually have enough work for all the threads.